universität innsbruck

BlockQuiz: A Web-Based Block Programming Platform with Auto-Grading

Bachelor Thesis Final Presentation

Fabio Plunser

Supervisor: Ass. Prof. Dr. Michael Vierhauser
Quality Engineering Research Group

Outline

1. Motivation & the gap
2. Requirements
3. What BlockQuiz is + live demo
4. How it works: architecture, grading, classes
5. Pedagogical design rationale
6. Evaluation: technical & with teachers
7. Lessons learned, limitations & future work

Motivation & the gap

Digitale Grundbildung is mandatory in Austrian schools since 2022/23, but the available block tools do not fit classroom constraints [1].

- Scratch – creative, open-ended, no deterministic grading
- Code.org – well scaffolded, but cloud-only
- MakeCode – great for hardware, less for assessment
- Blockly – a library, not a platform on its own

The gap: short, gradable exercises a teacher can author, with a privacy-conscious deployment that fits a school network.

Requirements

- Teachers author exercises and courses without coding
- Learners get immediate, understandable feedback
- Automatic grading, plus a hint system for support
- Multiple exercise types
- Bilingual (DE/EN) UI and content
- Safe execution of learner code

BlockQuiz in one slide

A web platform for short block-programming exercises:

- Three exercise types: `io`, `turtle`, `robot`
- Per-exercise constrained Blockly toolbox
- Browser sandbox + authoritative server-side regrading
- Teacher CMS, class system with IdP sync, cohort analytics
- Self-hosted, SQLite, no required external services

Built with SvelteKit, Svelte 5, Blockly, Bun, Drizzle ORM, Better Auth.

Demo Screencapture

https://universe.uibk.ac.at/s/nJ2wHSaeL4GBkNc

How it works: dual-path execution

Every submission is executed twice, by design.

Browser path (immediate)

- Hidden iframe, blocked globals
- Time / step / loop limits
- learner sees feedback now

Server path (authoritative)

- Isolated Bun subprocess, `node:vm`
- No DB, no session, hard kill timeout
- the stored score, not the client’s

Grading & publish validation

One result shape for all types: pass/fail, score, per-test feedback.

- `io`: normalised string compare (trim, whitespace, case)
- Visual: command-trace replay, not pixels, deterministic

Publish gate blocks broken exercises: bilingual content, parseable starter XML, block ids in toolbox, ≥ 1 test, and BFS reachability for turtle/robot.

Class system & identity-provider sync

Cohorts are first-class and can sync to the school IdP.

- Two membership sources per row: manual (admin) and sso (IdP)
- On every SSO login, IdP rows are reconciled, manual rows untouched
- Course assignment by class

Roster stays in sync with the school IdP without a separate SCIM endpoint.

Pedagogical design rationale

Each design choice is anchored in learning theory:

- Per-exercise toolbox $\rightarrow$ Cognitive Load Theory [2]
- Staged, captured hints $\rightarrow$ Zone of Proximal Development [3]
- Replay + plain-language readout $\rightarrow$ debugging as a computational thinking skill [4], blocks-to-text bridge [5]

Each one is captured per attempt and surfaced in analytics.

Technical evaluation

296 tests across 27 files, all passing.

Area	Tests
Canvas engines (turtle, robot, grid, pathfinding)	84
Graders (turtle, canvas, io normalisation)	45
Sandbox + authoritative subprocess execution	23
Class system + SSO (sync, enrolment, OIDC & SAML)	97
Analytics, achievements, readout, i18n, publish gate	∼47

User evaluation – method

Small qualitative probe, n = 2 student teachers at UIBK.

- Four structured tasks with open-text responses
- 20-item questionnaire (1–5), short debrief afterwards
- P1: student teacher, some digital-learning experience
- P2: kindergarten teacher, almost no software-platform experience

Not a controlled study; the value is in the themes that emerged.

User evaluation – results

Means: P1 = 4.5, P2 = 4.65 (of 5).

- Highest: suitability for children, age-appropriateness, feedback usefulness, would recommend after improvements
- Lowest: ”I could orient myself quickly”, ”feels intuitive”

Pedagogical fit is recognised; onboarding is the weak spot.

User evaluation – themes & actions

What worked

- Clean, inviting UI and layout
- Recognised pedagogical fit
- Home in Digitale Grundbildung / MINT

Weak spots (addressed)

- CMS onboarding overload
- DE Blockly labels (fixed)
- Small bugs (fixed)

Debrief: “Once the bugs are fixed, both would happily use it in class.” None of the issues were architectural.

Lessons learned

- Typed discriminated-union exercise model: keeps CMS, player, sandbox, analytics honest as the system grew
- Message-based sandbox protocol, made subprocess regrading a small addition, not a refactor
- Publish-validation gate, caught real authoring mistakes during development
- Using SvelteKit experimental features, remote functions and async svelte

Limitations & future work

Limitations of this thesis

- User study: $n = 2$, student teachers only, no child participants
- Exercise type addition requires quite extensive code changes, but has been simplified as much as possible

Future work

- Classroom pilot with children in the 8-12 target range
- Blocks-to-text showing the code for students to learn code.

Conclusion

BlockQuiz shows that

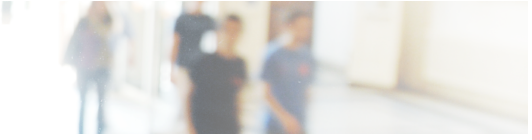
- constrained, deterministically-graded block exercises,
- teacher authoring with publish validation,
- identity-provider-synchronised classes & analytics, and
- privacy-conscious, self-hosted deployment

can be one coherent system, and that two student teachers see it as something they would use in their own future classrooms once the remaining polish is done.

Thank you

Questions?

https://git.uibk.ac.at/csba3598/BlockQuiz
https://thesis.fabioplunser.com



References I

[1] Mit den Digitalisierungsinitiativen und Projekten, „Schule 4.0“, „Digitale Grundbildung“, „Denken lernen, Probleme lösen“ gibt es seitens des Bundesministeriums für Bildung, Wissenschaft und Forschung Konzepte, die sich über die gesamte Schullaufbahn strecken. URL: https://appinventor.mit.edu/.

[2] John Sweller. “Cognitive load during problem solving: Effects on learning”. In: Cognitive Science 12.2 (1988), pp. 257–285. DOI: 10.1207/s15516709cog1202_4.

[3] Lev S. Vygotsky. Mind in Society: The Development of Higher Psychological Processes. Harvard University Press, 1980.

References II

[4] Karen Brennan and Mitchel Resnick. “New frameworks for studying and assessing the development of computational thinking”. In: Proceedings of the 2012 annual meeting of the American Educational Research Association. 2012.

[5] David Weintrop and Uri Wilensky. “To block or not to block, that is the question: students’ perceptions of blocks-based programming”. In: Proceedings of the 14th International Conference on Interaction Design and Children (IDC ’15). 2015, pp. 199–208. DOI: 10.1145/2771839.2771860.